

WYDZIAŁ INFORMATYKI



Ćwiczenie laboratoryjne

Transmisja szeregową w systemach mikroprocesorowych

Nazwa kodowa: **serial**. Wersja **20260104**

Ada Brzoza-Zajęcka, Wojciech Zaborowski

abrzoza@agh.edu.pl , zab@agh.edu.pl

1 Wstęp

1.1 Cel ćwiczenia

Celem ćwiczenia jest zapoznanie się z działaniem portów szeregowych, w szczególności asynchronicznych, używanych w systemach mikroprocesorowych ogólnego przeznaczenia oraz wbudowanych i IoT.

1.1.1 Dlaczego warto spróbować nauczyć się czegoś realizując to ćwiczenie?

Porty szeregowe zbudowane nad typowymi interfejsami UART (*Universal Asynchronous Receiver-Transmitter*) były używane nieprzerwanie od kilkudziesięciu lat i nie zapowiada się, że szybko odejdą w zapomnienie. Pierwotnie były używane do komunikacji terminalowej (stąd nawet nazwa aplikacji typu "serial terminal"). Później szybko przyjęły się w przemyśle. Tam używane są zarówno na takiej warstwie fizycznej, jaką poznajemy w niniejszym ćwiczeniu (RS-232), jak i na znacznie bardziej odpornej na zakłócenia warstwie fizycznej RS-485. W warunkach przemysłowych wcześniej lub później trafimy na interfejs RS-485 zbudowany w oparciu również o UART i służący do komunikacji protokołem Modbus. Dokładnie ten sam interfejs UART zastosowany z jeszcze innymi warstwami fizycznymi znalazł mnóstwo zastosowań w przemyśle muzycznym i rozrywkowym, np. do przesyłania cyfrowej informacji o dźwiękach w standardzie MIDI. Standard MIDI, choć powstał w latach '80 XX wieku, to nawet dziś nie zapowiada się, że szybko nas opuści. Interfejs DMX512 bazuje na interfejsie UART, tylko z nieznacznie rozbudowaną sygnalizacją i z warstwą fizyczną RS-485. DMX512 jest powszechnie wykorzystywany do sterowania światłami, urządzeniami, a nawet efektami pirotechnicznymi na scenach w trakcie przedstawień i koncertów. A z kolei w IoT i systemach wbudowanych powszechnie używa się interfejsu UART jako prostego sposobu na wyświetlanie komunikatów diagnostycznych i interakcję z urządzeniami.

Obecnie często możemy spotkać się z przesyłaniem danych "jak po UART" jednak przez USB - i tutaj warto poznać choćby pobieżnie klasę urządzeń USB-CDC (Communication Device Class), dzięki której możemy utworzyć urządzenie, które będzie widoczne w systemach jako "port szeregowy" jednak wirtualny, utworzony nad interfejsem USB - to zagadnienie także poruszamy w ćwiczeniu.

Dlatego, niezależnie czy przyjdzie nam pracować z systemami informatycznymi wbudowanymi, w branży IoT, z maszynami, z układami automatyki, w przemyśle czy to produkcyjnym, czy rozrywkowym - to jest ogromna szansa, że spotkamy się z interfejsem UART w jakiejś postaci.

1.2 Stanowisko testowe

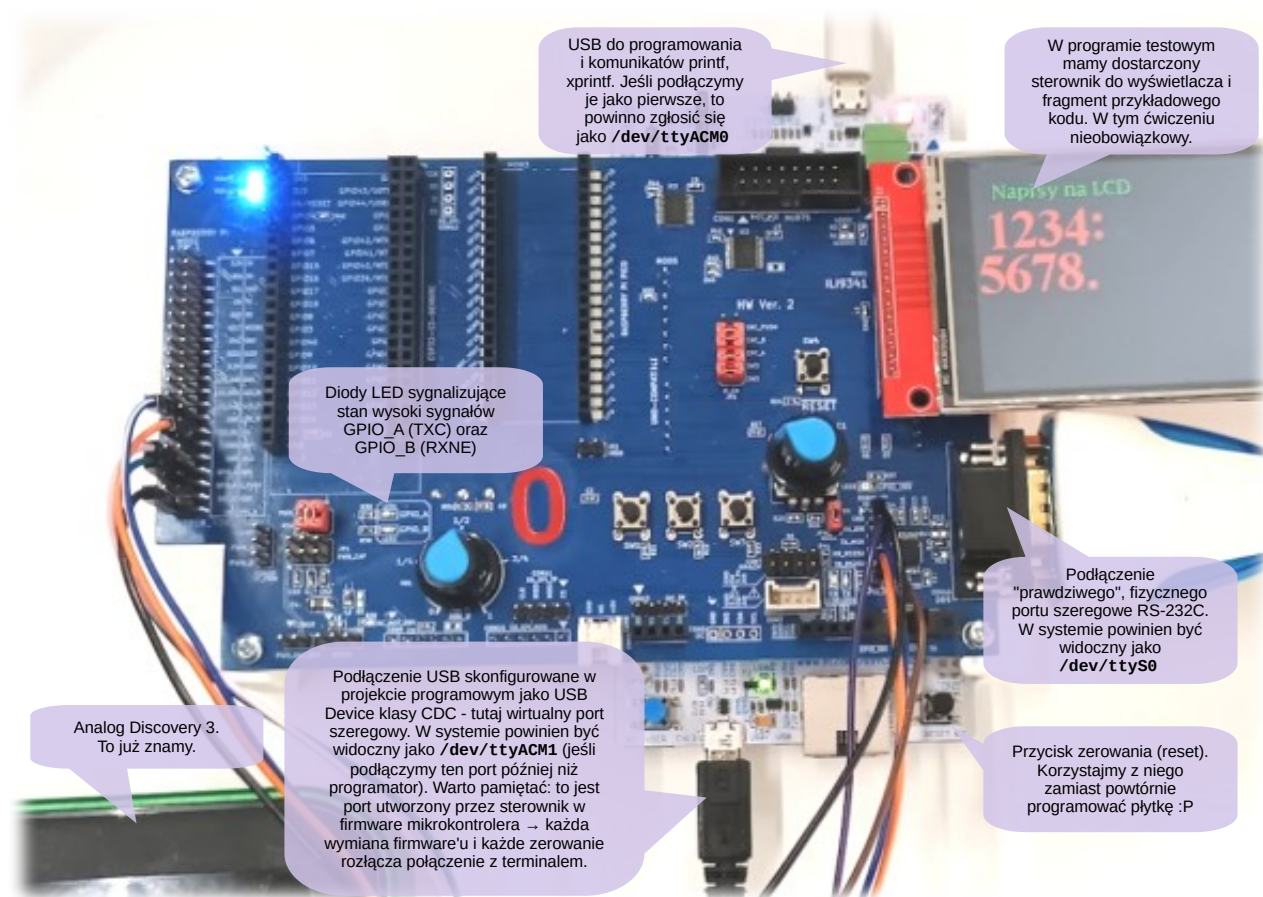
Ćwiczenie przeprowadzone jest z wykorzystaniem platformy sprzętowej **STM32 Nucleo-F429 (lub -439)** z mikrokontrolerem z STM32F429/439 z rdzeniem ARM Cortex-M4. Do realizacji ćwiczenia będzie potrzebna także dodatkowa płytki bazowa z niezbędnymi do realizacji ćwiczenia układami peryferyjnymi. Przy okazji mamy także do dyspozycji moduł graficznego wyświetlacza LCD o rozdzielczości 320x240 pikseli wyposażony w układ scalony sterownika ILI9341.

Od strony programowej użyjemy zintegrowanego środowiska:

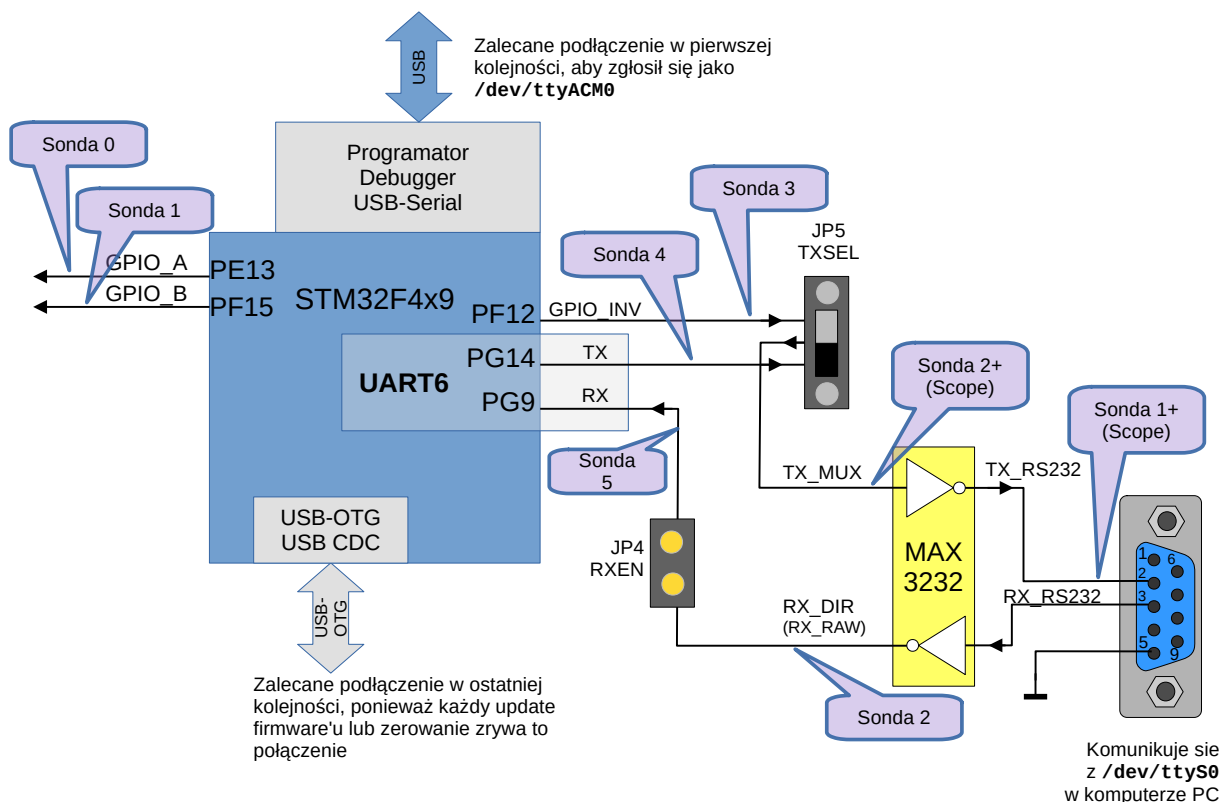
STM32CubeIDE **najlepiej w wersji 1.19**. (Inne wersje także mogą działać, ale nie jest to gwarantowane, ponieważ projekt startowy powstał na 1.19.0)

Do obserwacji przebiegów użyjemy Digilent WaveForms.

Poniżej przedstawiono widok oraz uproszczony schemat blokowy stanowiska.



Uproszczony schemat blokowy interfejsu pomiędzy mikrokontrolerem a urządzeniem komunikującym się w standardzie RS-232C:



1.3 Materiały pomocnicze

Przed przystąpieniem do realizacji ćwiczenia warto zapoznać się z treścią wykładów, w szczególności:

- Interfejsy komunikacyjne w systemach wbudowanych i IoT,
- Wyjątki i przerwania,
- DMA.

Oczywiście dokumentacja mikrokontrolera STM32F429/F439 także bardzo pomoże.

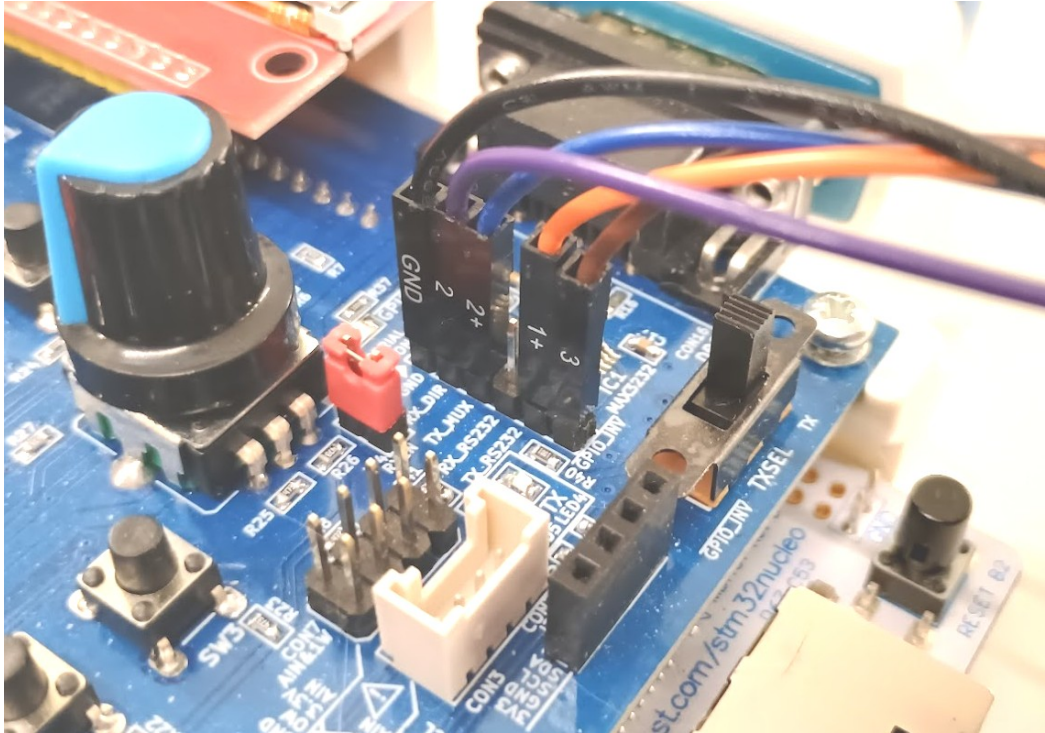
2 Przygotowanie

Ćwiczenie wykonywane jest na unikalnej płytce testowej. Potrzebne fragmenty dokumentacji są udostępnione na platformie UPEL. Do wykonywania pomiarów użyjemy uniwersalnego modułu Analog Discovery 3 (AD3) produkcji Digilent. Jego sondy pomiarowe podłączamy według opisu w dalszej części instrukcji. Po wykonaniu ćwiczenia zalecane jest, aby nie odłączać sond.

Jeśli zastaniemy stanowisko z już podłączonymi sondami, to mimo wszystko warto zapoznać się z dalszą częścią opisu ich podłączania, aby zrozumieć, co tak naprawdę mierzymy ;)

2.1 Podłączenie sond AD3 do interfejsu UART

Podłączenie sond AD3 do sygnałów łącza UART (CON15 na płytce) zostało przedstawione na poniższej fotografii oraz w tabeli. Proszę zwrócić uwagę, że tym razem mamy rozróżnienie na sygnały analogowe (te z +: 1+ i 2+) oraz cyfrowe (sondy cyfrowe 2 i 3).



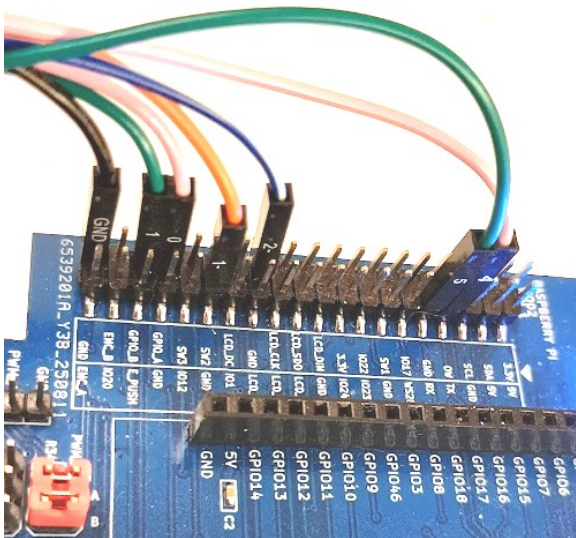
CON15	Nazwa	Sonda	Opis
6	GPIO_INV	3 (Cyfrowa)	sygnał GPIO mikrokontrolerów, który możemy skierować do wyjścia TX fizycznego łącza RS232-C, gdy przesuniemy przełącznik TXSEL w lewo
5	TX_RS232	1+ (Analogowa)	Wyjście TX fizycznego łącza RS232-C - napięcia zgodne ze standardem (za transceiverem)
4	RX_RS232	-	Wejście RX fizycznego łącza RS232-C - napięcia zgodne ze standardem (za transceiverem)
3	TX_MUX	2+ (Analogowa)	Sygnał TX zgodny z poziomami logicznymi mikrokontrolerów na płytce (przed transceiverem)
2	RX_DIR (RX_RAW)	2 (Cyfrowa)	Sygnał RX zgodny z poziomami logicznymi mikrokontrolerów na płytce (przed transceiverem)
1	GND	GND	Masa

2.2 Podłączenie sond AD3 do GPIO (sygnały pomocnicze)

Podłączenia sond dla sygnałów GPIO oraz połączenia masy, w tym sond analogowych referencyjnych (z "-") zostały przedstawione analogicznie. W rzeczywistości sygnały 1- i 2- mogą być napięciami odniesienia dla modułu oscyloskopu cyfrowego, wbudowanego w AD3 i w ten sposób korzystamy w nich w ćwiczeniu. Nie używamy jednak pomiarów różnicowych, więc podłączamy je do dowolnie wybranych pinów masy, tak jak sondy oznaczone GND. Wystarczy podłączyć nawet jedną sondę GND do wybranych pinów masy, jeśli nie potrzebujemy "pięknych" przebiegów.

Wygodne może być skorzystanie ze złącza dla Raspberry Pi po lewej stronie płytki testowej. Tutaj ważne jest miejsce podłączenia sond cyfrowych kanałów 1, 2, 4 i 5. Natomiast kabelki GND oraz "1-" i "2-" podłączamy do dosłownie byle której dostępnej masy.

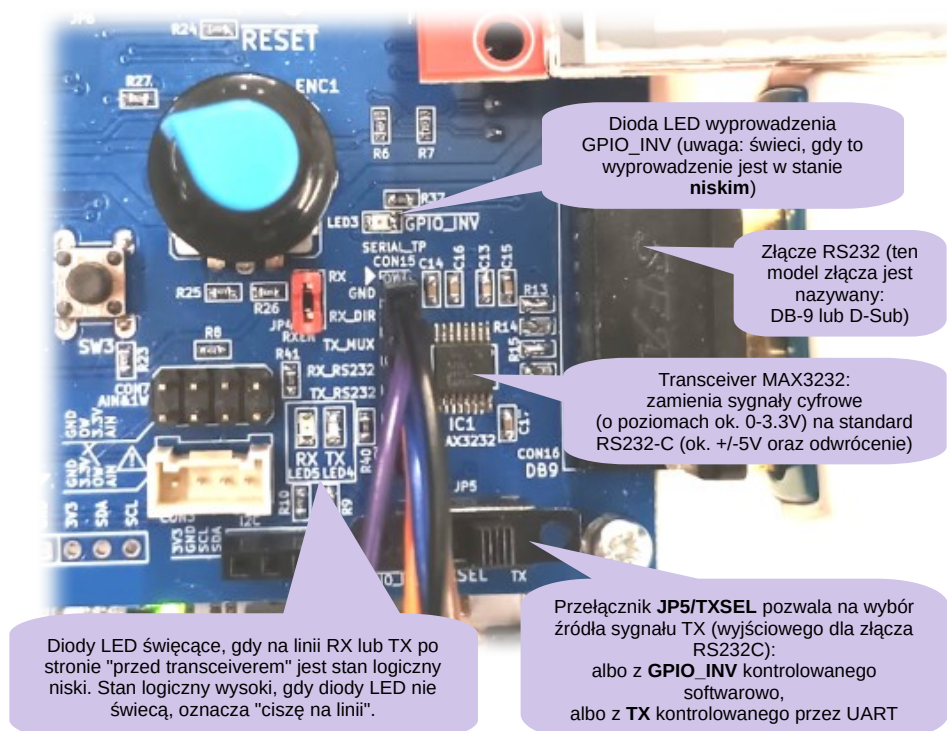
Warto wiedzieć, że na tym etapie podłączamy także dodatkowe sondy cyfrowe do sygnałów RX i TX. Sondy dla sygnału RX są właściwie zdublowane: przy założonej zworze RXEN można ten sygnał mierzyć zarówno sondą cyfrową 2 jak i 5. Natomiast bardzo istotne jest tutaj przede wszystkim doprowadzenie sygnału TX do sondy cyfrowej 4. Pozwoli to na obserwację zachowania TX względem innych sygnałów interfejsu UART w widoku analizatora stanów logicznych.



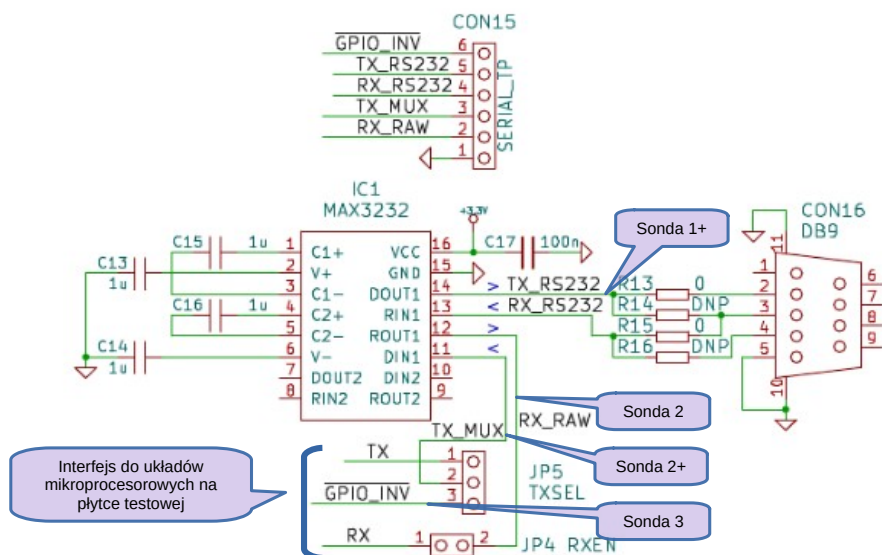
Sygnał MOD2	Sonda	Opis
GND	GND	Masa
GPIO_A	0 (Cyfrowa)	Wejście analizatora stanów logicznych podłączone do wyjścia GPIO_A
GPIO_B	1 (Cyfrowa)	Wejście analizatora stanów logicznych podłączone do wyjścia GPIO_B
TX	4 (Cyfrowa)	Dodatkowa sonda cyfrowa dla sygnału TX
RX	5 (Cyfrowa)	Dodatkowa sonda cyfrowa dla sygnału RX
GND	1-	Wejścia napięć odniesienia oscyloskopu wbudowanego w AD3
GND	2-	

2.3 Fragment płytki odpowiedzialny za transmisję szeregową

Fragment płytki, któremu szczególnie przyjrzymy się w niniejszym ćwiczeniu, został przedstawiony i pokrótce objaśniony na fotografii poniżej.



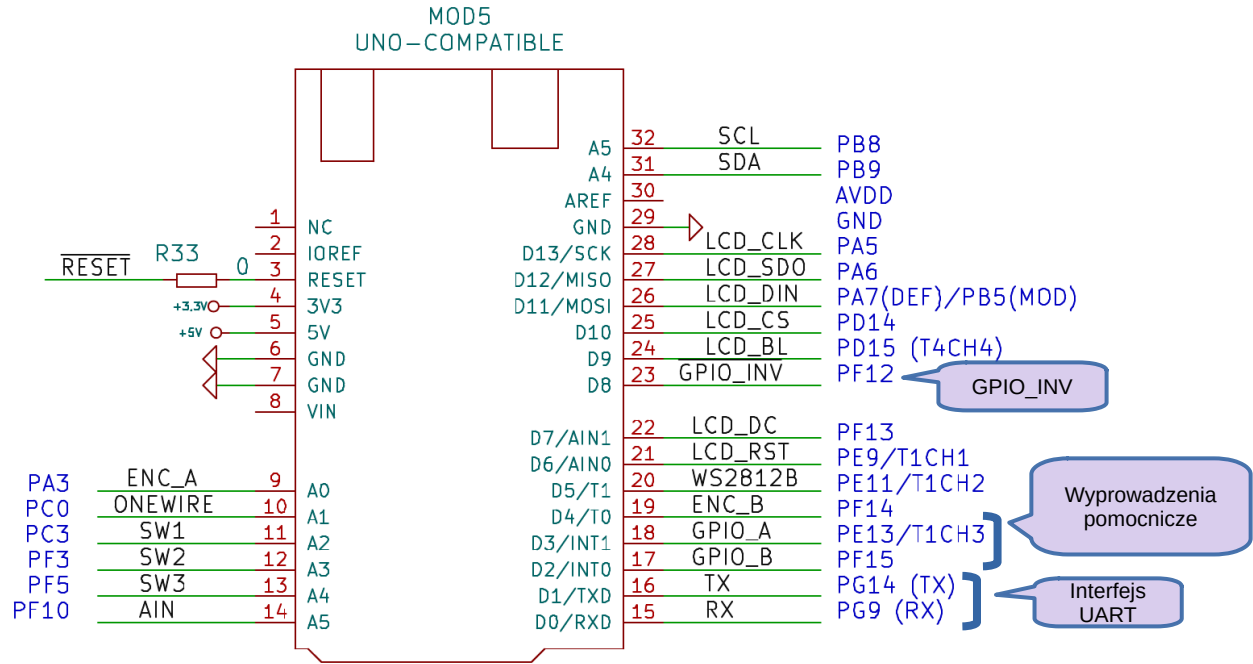
Dokładny schemat elektryczny podsystemu komunikacji, którym będziemy zajmować się w ćwiczeniu został zamieszczony poniżej.



Rezystory (właściwie zwory) R13-R16 pozwalają na wybór, czy na płytce testowej zostanie wlutowane gniazdo czy wtyk dla łącza RS232. W aktualnej wersji jest tam wlutowane gniazdo, dlatego sygnały są kierowane przez zwory R13 i R15 (0 Ω), natomiast R14 i R16 oznaczone są jako DNP od *Do Not Populate*, co jest często stosowanym akronimem sugerującym "nie montować".

Sygnały TX, RX i GPIO_INV są skierowane do złączy wszystkich modułów znajdujących się na płytce testowej. W przypadku używanego w ćwiczeniu STM32F429-Nucleo, płytka ta dołączona

jest przez interfejs kompatybilny z klasycznym Arduino Uno wg schematu zamieszczonego poniżej.

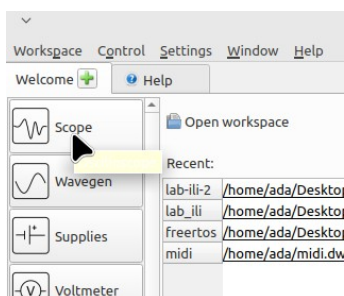


2.4 Konfiguracja WaveForms

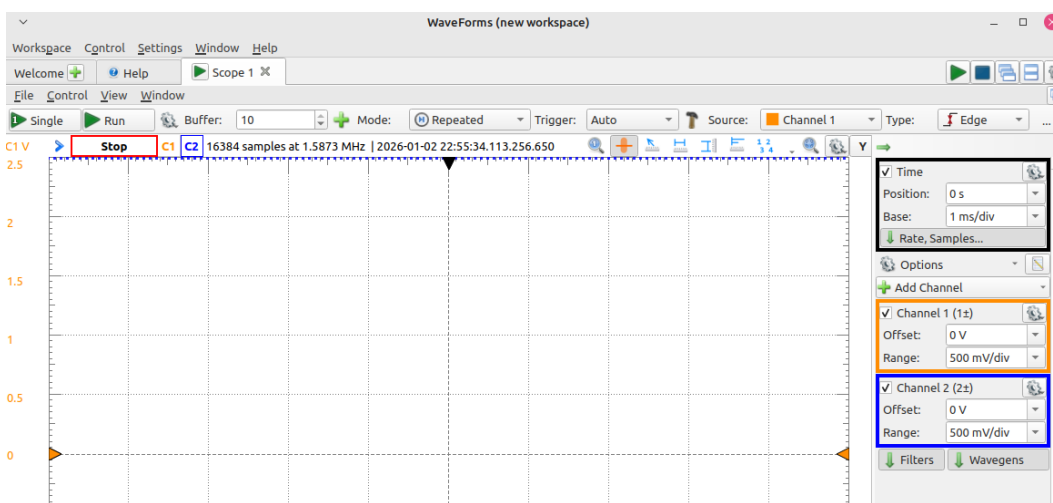
Gdy analizator AD3 jest już przygotowany do pracy, a sondy zostały podłączone wg opisów w punktach 2.1 i 2.2, otwieramy WaveForms. Po zakończeniu konfiguracji WaveForms można **zapisać** pole robocze (*workspace*) tej aplikacji, aby można było łatwiej wrócić do realizacji ćwiczenia na kolejnych zajęciach.

2.4.1 Konfiguracja oscyloskopu

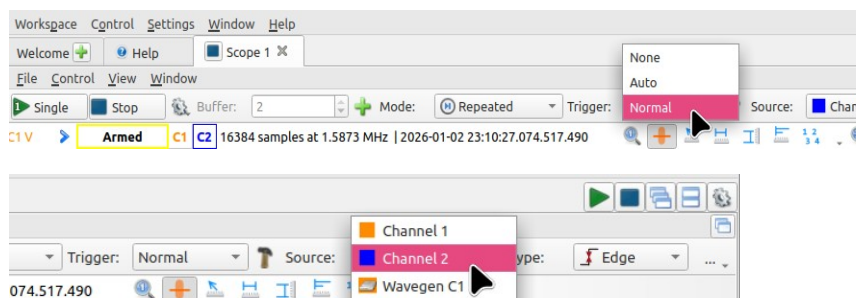
W WaveForms nawigujemy najpierw do oscyloskopu: z listy przyrządów wybieramy Scope (z jakiegoś powodu przy tworzeniu zrzutów ekranu nie wyświetlały się poprawnie opisy - stąd puste labelki ;)):



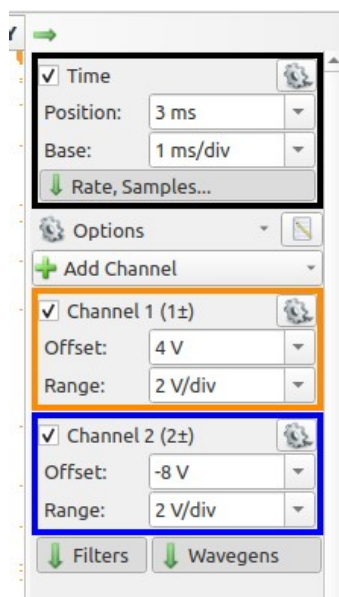
Pojawi się zakładka Scope 1, specyficzny pasek narzędzi oraz obszar kreślenia.



Przestawiamy metodę wyzwalania (**Trigger**) na **Normal**, a jako źródło sygnału wyzwalania pozostawiamy lub wybieramy kanał 2 (**Channel 2**). Typ wyzwalania ustawiamy/pozostawiamy **Edge** co oznacza wyzwalanie zboczem narastającym.



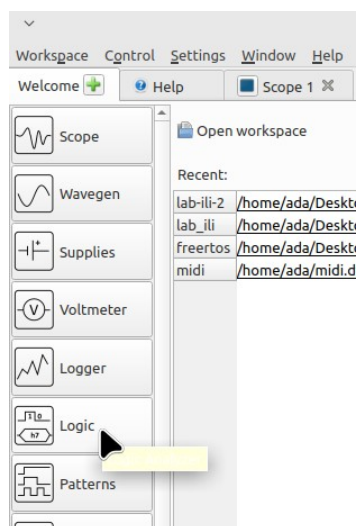
Możemy także wstępnie skonfigurować położenie wykresów oraz skalę na osi czasu i napięć. Na dobry początek można spróbować takich ustawień:



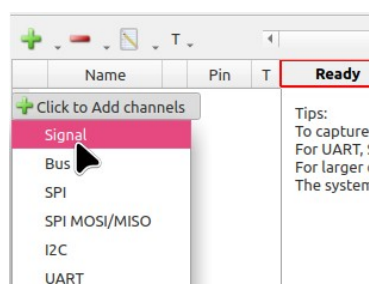
Oczywiście nie ma sensu sztywno trzymać się tych ustawień i, gdy przejdziemy do rzeczywistej obserwacji sygnałów, można je dynamicznie i interaktywnie zmieniać w taki sposób, aby można było zobaczyć jak najwięcej interesujących fragmentów przebiegów.

2.4.2 Dodawanie kanałów cyfrowych

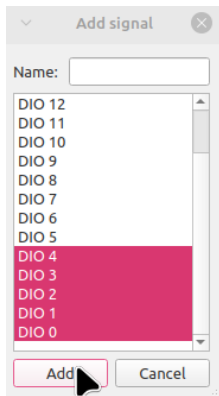
Nie zamykamy zakładki Scope - przyda nam się za chwilę. Wracamy do zakładki *Welcome* i wybieramy *Logic*.



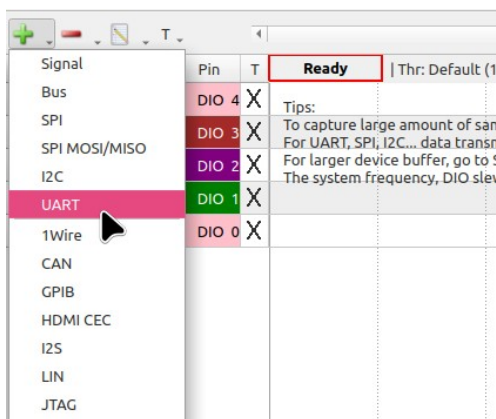
Następnie - podobnie jak poprzednich ćwiczeniach - dodajemy sygnały.



Wystarczą sygnały 0-4:



Możemy także dodać ponownie część sygnałów, jednak tym razem z analizą protokołów - może to przydać się w obserwacji i interpretacji przebiegów. I tak, możemy dodać sygnały UART:

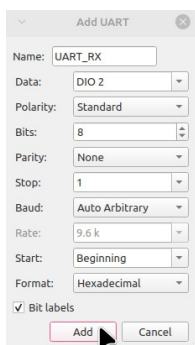


Warto nazwać te sygnały opisowo:

UART_RX na DIO2 (sonda 2),

GPIO_INV na DIO3 (sonda 3),

UART_TX na DIO4 (sonda 4).



Warto także zmienić *Format* na szesnastkowy (**Hexadecimal**) lub szesnastkowy+ASCII (**HexASCII**). HexASCII wyświetla zarówno przesyłaną liczbę szesnastkową jak i jej interpretację w kodzie ASCII. Nie zawsze jednak może być wygodny - szczególnie, gdy chcemy wyświetlać dużo informacji na jednym ekranie (przyczyna jest prozaiczna: każdy interpretowany znak zajmuje trochę więcej miejsca).

Oprócz tego, nadanie nazw sygnałom DIO0...DIO4 także może być dobrym pomysłem, aby łatwiej je rozpoznawać. Przykładowo:

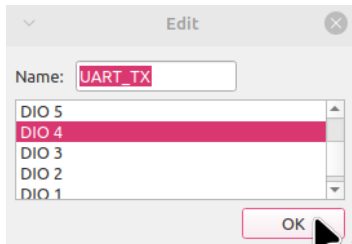
DIO0 → GPIO_A

DIO1 → GPIO_B

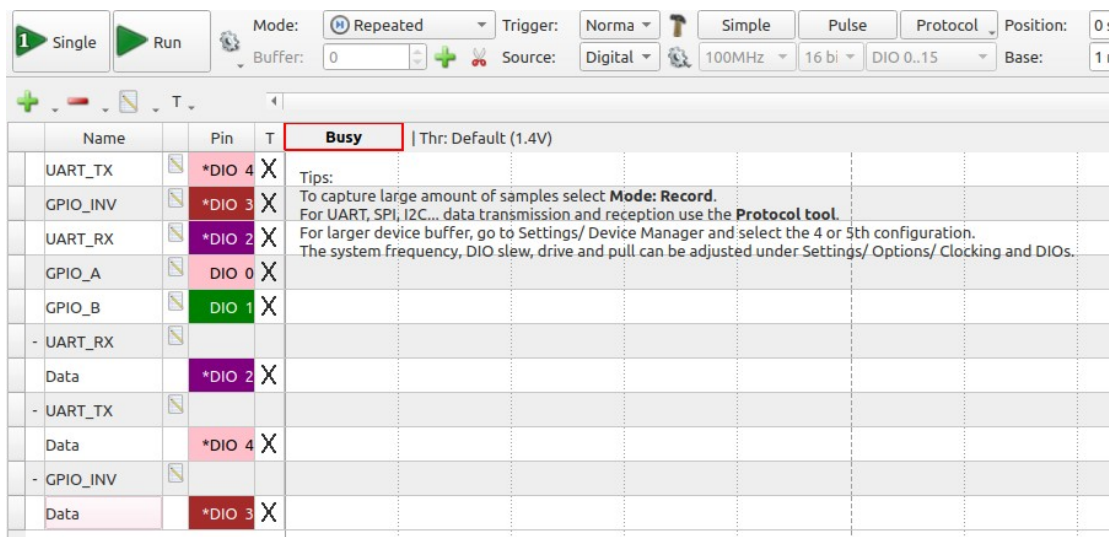
DIO2 → UART_RX

DIO3 → GPIO_INV

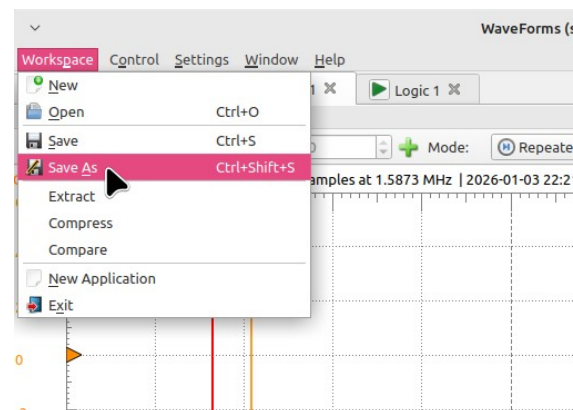
DIO4 → UART_TX



Tym sposobem fragment obszaru kreślenia może wyglądać mniej więcej tak:



Tak jak wspomniano wcześniej, **warto zapisać pole robocze WaveForms**, aby można było szybciej wrócić pomiarów, szczególnie jeśli planujemy kończyć wykonanie ćwiczenia na kolejnych zajęciach



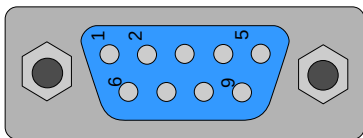
3 Praca z oprogramowaniem i pomiary

W tej części przejdziemy do tworzenia oprogramowania.

3.1 Biegłość w radzeniu sobie z portami szeregowymi

Zanim zaczniemy, warto opanować rozpoznawanie portów szeregowych pod systemem Linux. Możemy spotkać się z trzema urządzeniami znakowymi reprezentującymi porty:

- **/dev/ttySn**, np. **/dev/ttyS0** - jeśli urządzenie zgłasza nam się w ten sposób, to prawdopodobnie jest to port szeregowy wbudowany w komputer - taki z klasycznym, najczęściej 9-pinowym złączem. Obecnie jest już spotykany bardzo rzadko w komputerach domowych czy zwykłych biurowych, za to dalej powszechny w komputerach przemysłowych, a czasem także można go znaleźć w desktopowych komputerach biznesowych wyższej klasy. Komputery w laboratorium mają taki port i użyjemy go w ćwiczeniu. To ten, który mamy podłączony przewodem z prawej strony płytki przy pomocy "tego ogromnego złącza" (ogromnego przynajmniej jak na współczesne czasy i elektronikę konsumencką ;)).



- **/dev/ttyUSBn**, np. **/dev/ttyUSB0**: tak zgłaszają się "prześciówki" (adaptery) portu szeregowego tworzonego przy pomocy interfejsu USB. Są to urządzenia USB specyficzne dla poszczególnych producentów, ale w efekcie i tak otrzymujemy w systemie wirtualne urządzenie, które zachowuje się jak port szeregowy.
- **/dev/ttyACMn**, np. **/dev/ttyACM0**: tak zgłaszają się urządzenia klasy USB CDC czyli Communication Device Class. Pierwotnie klasa ta miała służyć przy komunikacji przez modemy, jednak została zaadaptowana do komunikacji z urządzeniami wbudowanymi, gdzie istotne jest zachowanie kompatybilności. Najczęściej spotykamy się z takimi urządzeniami, jeśli są one zbudowane nie o specjalizowany układ scalony (jak te ttyUSB), ale w oparciu o mikrokontroler ogólnego przeznaczenia.

Ciekawostka 1: pod systemami Windows wszystkie wyżej wymienione porty zgłaszają się jako COMx (COM1, COM2, ...). Porty "sprzętowe", jeśli są obecne, to w Windowsie znajdziemy je prawdopodobnie pod niższymi numerami, np. COM1. Natomiast porty "z prześciówek" dostają wyższe numery, typowo COM5 i kolejne.

Ciekawostka 2 (ale ta jest **bardzo ważna przy realizacji ćwiczenia**): w systemach Linux dołączane porty szeregowo na USB tego samego typu (np. ACM) zgłaszają się pod kolejnymi numerami licząc od 0 i - uwaga - w kolejności podłączania. A jeśli zostaną podłączone przy starcie systemu, mogą pojawić się w systemie zależnie od tego, do których portów USB zostały włączone.

Dlatego...

Przewidywalnym i zalecanym sposobem sprawdzenia, które urządzenie USB generuje nam który

port, jest najpierw odłączenie wszystkich urządzeń tworzących porty szeregowo. A następnie podłączanie kabli USB "po jednym", sprawdzając po podłączeniu każdego, co nowego pojawiło się w systemie, przy pomocy komendy:

```
ls -l /dev/ttyACM*
```

Dzięki niej dostaniemy listę aktualnie utworzonych portów ttyACM w systemie. Porty, które były wcześniej podłączone, będą miały takie numery, jakie miały przydzielone po podłączeniu. Natomiast nowe porty, dostaną wyższe numery (właściwie pierwsze wolne).

Jeśli pogubimy się w numeracji portów, można:

1. odłączyć od komputera wszystkie urządzenia, które podejrzewamy, że mogą się zgłaszać jako porty szeregowo na USB; przy realizacji ćwiczenia na pewno będą to: zarówno złącze USB-OTG znajdujące się na dole płytki, jak i programator płytki Nucleo (złącze USB w części wyglądającej jakby wołała "odłam mnie", czego jednak nie robimy)
2. odczekać 5-10 sekund - w tym czasie urządzenia ttyACM powinny "zniknąć" z systemu, co powinniśmy zobaczyć wydając kolejny raz komendę `ls -l /dev/ttyACM*`
3. podłączyć ponownie: najpierw (to ważne) programator na górze płytki,
4. później USB-OTG (dół płytki).

3.2 Projekt bazowy i rozpoczęcie pracy

Na platformie UPEL w sekcji ćwiczenia "serial" mamy dostępny projekt bazowy. Jak zwykle: ściągamy go do swojego katalogu o unikalnej nazwie, rozpakowujemy i otwieramy bądź importujemy w środowisku STM32CubeIDE w wersji 1.19.0.

Możemy wykonać próbną kompilację , aby sprawdzić, czy nie pojawiają się błędy lub niepokojąco wyglądające ostrzeżenia.

3.2.1 Nawiązanie podstawowej komunikacji

Podłączamy programator płytki Nucleo do komputera (część "odłam mnie"). Złącze **USB-OTG na dole płytki pozostawiamy na tym etapie odłączone od komputera**. Rozpoznajemy-potwierdzamy, który port to programator:

```
ls -l /dev/ttyACM*
```

Dalej w instrukcji zakładamy, że jest to ttyACM0.

Otwieramy - uwaga - tym razem aż dwa terminale putty.

Jednym terminalem putty łączymy się z portem wygenerowanym przez programator (pewnie **/dev/ttyACM0**) na szybkości **115200 b/s** z typowymi parametrami 8N1 (domyślne).

Drugim terminalem putty łączymy się z portem szeregowym **/dev/ttyS0** na szybkości **2400 b/s** - będzie to port szeregowy sprzętowy dostępny w komputerze i będziemy się z nim komunikować przez port UART6 mikrokontrolera.

3.2.2 Zapoznanie się z bazowym oprogramowaniem

Z oprogramowaniem bazowym komunikujemy się przez główny terminal wysyłając do niego znaki z klawiatury. Na terminalu będziemy widzieć komunikaty wypisywane przez firmware mikrokontrolera przy pomocy funkcji `printf`, `xprintf` oraz ew. wielu innych dostępnych funkcji z pliku `dbgu.c`.

Znaki wysłane z terminala do oprogramowania są interpretowane w funkcji `terminal_iface`:

```
void terminal_iface(void){
    uint8_t key = inkey();
    switch(key){
        case 0:
            break;
        case 'u':
            CDC_Transmit_FS((uint8_t*)"abc ", 4);
            break;
        case 'r':
            char rxed_chr = uart_rx_polling();
            if(rxed_chr){
                xprintf("UART6 rxed sth: %02X = %c\n", (unsigned int)rxed_chr, rxed_chr);
            }
            break;
        case '1':
        case '2':
        case '3':
        case '4':
            int count = key - '0';
            const char tx_chr = 0x35;
            xprintf("sending %d chars of value %02X (%c)... ", count, (int)tx_chr, tx_chr);
            for(int i=0; i<count; i++){
                uart_tx_polling(tx_chr);
            }
            xprintf("done\n");
            break;
        default:
            uart_tx_polling(key);
            break;
    }
}
```

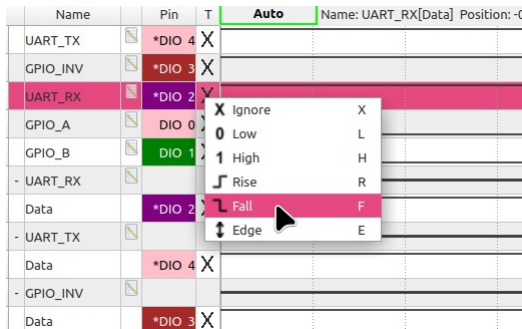
- Gdy wyślemy jakiś znak z klawiatury przez terminal, to ten znak zostanie odebrany w funkcji `inkey` (z `dbgu.c`). Następnie, jeśli odebrany znak ma kod inny niż 0, to zostanie sprawdzony, czy nie odpowiada któremuś z warunków w bloku switch-case.
- Na początek interesuje nas wysyłanie znaków 'r' oraz '1'-'4'. Wysłanie znaku 'r' (od receive) ma spowodować wyświetlenie ostatnio odebranego znaku z portu szeregowego UART6 mikrokontrolera w wywołaniu funkcji `uart_rx_polling`. Natomiast wysłanie jednego ze znaków '1'-'4' ma rozpocząć wysyłanie odpowiednio od 1 do 4 znaków zdefiniowanych w stałej `tx_chr` jak najszybciej jeden po drugim przez portu UART6 mikrokontrolera przez funkcję `uart_tx_polling`.
- Wysłanie dowolnego innego, nieobsługiwanego znaku spowoduje przekierowanie tego znaku na UART6 (blok `default`).

I tutaj pojawiają się pierwsze zadania:

Jak można łatwo przekonać się - funkcje `uart_rx_polling` i `uart_tx_polling` to "wydmuszki" - gdy do nich zajrzemy, to okażą się puste. I, w ramach realizacji pierwszych zadań, należy napisać ich treść. → → →

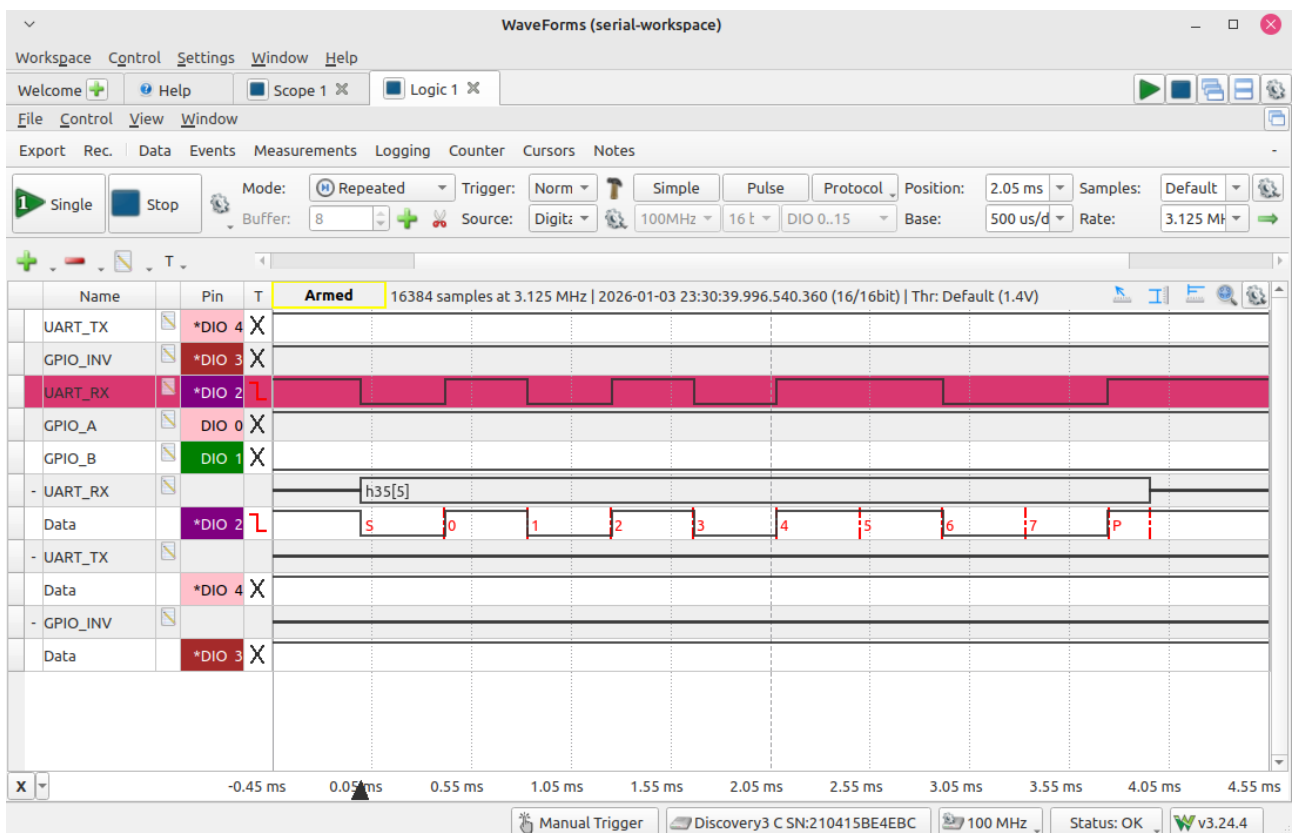
3.2.3 Odczyt danych z UART6 - piszemy treść uart_rx_polling

Warto zacząć od sprawdzenia, czy dane na linię RX w ogóle przychodzą. W tym celu uruchamiamy wcześniej przygotowany (pkt 2.4.2) analizator stanów logicznych (Logic) w WaveForms klikając **Run**. Na razie obserwujemy linię odpowiadającą sondzie 2 (*UART_RX*). Gdy spróbujemy wysłać coś z terminala, to prawdopodobnie na tej linii przebieg jedynie "błyśnie i ucieknie". Aby uchwycić transmitowanych znak, ustawiamy na niej wyzwalanie zboczem opadającym (**Fall**):



Spróbujmy ponownie :)

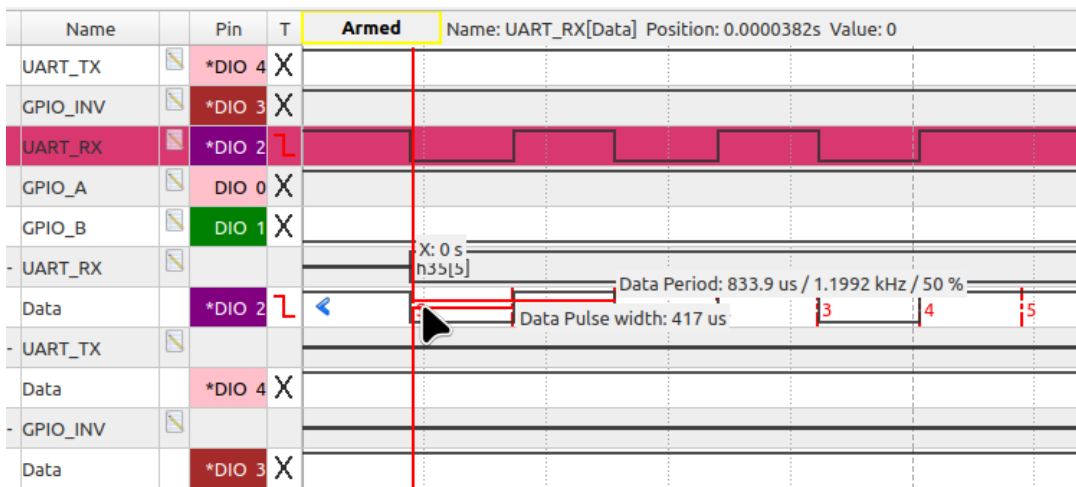
Po każdym wysłaniu kolejnego znaku zostaną na nowo uaktualnione zarejestrowane przebiegi na linii *UART_RX* oraz interpretacja znaku wykonana przez uprzednio przygotowany analizator protokołów. Po ewentualnym dostosowaniu sposobu wyświetlania, możliwe, że zobaczymy ekran podobny do poniższego (tutaj widzimy tryb wyświetlania HexASCII):



Możemy także zmierzyć czas trwania jednego bitu aby sprawdzić, czy w putty dobrze ustawiliśmy szybkość transmisji. Powinna ona wynosić 2400 bitów na sekundę. Możemy kliknąć tę ikonkę:



i przejść do trybu interaktywnego pomiaru przebiegu:



Tutaj powyżej widzimy, że skoro 2 bity trwają około 833,9 μ s (co odpowiada częstotliwości 1,1992 kHz), to znaczy, że jeden bit będzie trwał połowę tego czasu (ok. 416,95 μ s), co dałoby szybkość transmisji ok. 23984 bitów na sekundę. I rzeczywiście widzimy, że "Pulse width" trwa 417 μ s (czyli blisko tej wartości). W sytuacji idealnej jeden bit powinien trwać 1/2400 sekundy czyli ok. 0,4167 ms.

Jeśli do tej pory wszystko zgadza się, to przechodzimy do pisania treści funkcji `uart_rx_polling`. Funkcja ma działać podobnie jak dostępna już `inkey` z `dbgu.c`. Jeśli od ostatniego wywołania nie pojawił się żaden nowy znak o niezerowym kodzie, to funkcja zwraca zero. A jeśli od ostatniego wywołania pojawił się nowy znak - funkcja ma zwracać kod ASCII tego znaku. Zatem odnajdujemy w kodzie `main.c` przygotowaną, pustą funkcję `uart_rx_polling` i zaraz uzupełnimy jej treść.

```
/*
 * tries to receive a character from an initialized UART interface
 * if something has been rxd, returns its ASCII code
 * if nothing new arrived, returns 0
 * obviously, if 0 arrives, it will be indistinguishable from "nothing arrived"
 */
static char uart_rx_polling(){
    return 0;
}
```

Przy projekcie skonfigurowanym w STM32CubeMX interfejsy UART reprezentowane są przez obiekty (właściwie struktury danych) typu `UART_HandleTypeDef`. W przypadku projektu bazowego do ćwiczenia i układu UART6 jest to obiekt `huart6` utworzony w następujący sposób:

```
UART_HandleTypeDef huart6;
```

Ten obiekt został zainicjalizowany w funkcji

```
static void MX_USART6_UART_Init(void)
```

dzięki czemu jest od razu gotowy do użycia.

```

static void MX_USART6_UART_Init(void)
{
    //...
    huart6.Instance = USART6;
    huart6.Init.BaudRate = 2400;
    huart6.Init.WordLength = UART_WORDLENGTH_8B;
    huart6.Init.StopBits = UART_STOPBITS_1;
    huart6.Init.Parity = UART_PARITY_NONE;
    huart6.Init.Mode = UART_MODE_TX_RX;
    huart6.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart6.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart6) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN USART6_Init 2 */

    /* ... */
}

```

→ → →

Aby sprawdzić, czy pojawił się nowy znak, należy najpierw odczytać zawartość rejestru statusowego (SR) interesującego nas modułu UART lub USART. Możemy tutaj posłużyć się zainicjalizowanym obiektem, w naszym przypadku `huart6`:

```
uint32_t status = huart6.Instance->SR;
```

Następnie należy sprawdzić, czy ustawiony jest bit RXNE (*Receive Not Empty*). Mamy do tego nawet gotową makrodefinicję z maską bitową `UART_FLAG_RXNE`.

Jeśli tak, to znaczy, że przyszedł nowy znak i możemy go odczytać z rejestru DR (*Data Register*). W przeciwnym razie - nic nie przyszło i zwracamy zero:

```
if(status & UART_FLAG_RXNE){  
    return huart6.Instance->DR;  
}  
else{  
    return 0;  
}
```

Tak jak w komentarzu w kodzie, funkcja zwróci zero, jeśli odczytany znak będzie wynosił zero i nie pozwala na rozróżnienie tej sytuacji od braku odebrania znaku. Oczywiście można ją napisać w taki sposób, aby te sytuacje rozróżniać, jednak tutaj trzymamy się implementacji minimalnej.

Aby przetestować działanie funkcji:

- z terminala `ttyS0` (prawdziwy port szeregowy) wysyłamy jakiś znak z klawiatury,
- w `WaveForms` obserwujemy przebieg odpowiadający wysłanemu znakowi,
- na terminalu `ttyACM0` (debugger) wysyłamy znak 'r', aby wywołać funkcję odczytującą znak - sprawdzamy komunikat, czy znak odpowiada wysłanemu (jeśli nie, to może oznaczać problem z szybkością lub parametrami transmisji na terminalu do `ttyS0`),
- powtarzamy aż nam się znudzi.

Nieobowiązkowo możemy także dodać reakcję i kasowanie flagi ORE (*Overrun Error*) - analogicznie jak w przypadku funkcji `inkey`, której kod znajdziemy w `dbgu.c`.

Gdy napiszemy, skompilujemy i uruchomimy kod, to rejestrujemy efekty naszej pracy przy pomocy zrzutów ekranu, nagrań i kopii aktualnego stanu `main.c`. Tak utworzone materiały umieszczamy na UPEL pod czytelną nazwą.

Następnie przechodzimy do pokazania osobie prowadzącej, jak ładnie działają efekty naszej dotychczasowej pracy:

- wysyłamy znak z klawiatury przez `ttyS0`,
- pokazujemy na żywo pojawiający się na terminalu `ttyACM0` komunikat informujący o odebraniu znaku z `ttyS0`,
- pokazujemy na przebiegach w `WaveForms`, że znaki przepływają przez obwody na płycie i docierają do mikrokontrolera.

3.2.4 Wysłanie danych przez UART6 - pierwsza implementacja uart_tx_polling

Wysyłanie znaków zrealizujemy w funkcji `uart_tx_polling`.

```
static void uart_tx_polling(char chr){  
    //tu wpiszemy treść funkcji  
}
```

Jako jedyny argument `chr` funkcja przyjmuje liczbę, którą należy wysłać. Oczywiście liczba ta może odpowiadać czytelny znakom ASCII dla łatwiejszego wyświetlania na terminalu.

Aby w mikrokontrolerach STM32F429/439 wysłać znak, wystarczy go wpisać do rejestru DR modułu UART. W naszym przypadku mogłoby to wyglądać tak:

```
huart6.Instance->DR = chr;
```

Ale...

Musimy najpierw sprawdzić, czy rejestr DR jest gotowy na przyjęcie nowego znaku, ponieważ wysyłanie może trwać bardzo długo w porównaniu z szybkością działania mikrokontrolera. A jeśli wpiszemy więcej znaków do DR bez skonfigurowanych kanałów DMA, to znaki te przepadną.

Rejestr DR jest gotowy na przyjęcie nowych danych wtedy, gdy flaga TXE (*transmission buffer empty*) jest aktywna. W trybie polling, musimy po prostu poczekać na jej aktywność. Jeśli używamy tylko jednej funkcji wpisującej dane, możemy przyjąć dowolną konwencję: czy czekamy przed wpisaniem, czy czekamy po wpisaniu. Jednak bardziej efektywne może się okazać czekanie przed wpisaniem danych, ponieważ wtedy nie marnujemy czasu na oczekiwanie na gotowość, jeśli w najbliższym czasie nie mamy zamiaru wpisywać więcej znaków - najwyżej poczekamy przed wysłaniem kolejnych.

Jak pewnie pamiętamy, tryb polling jest typowo bardzo nieefektywnym, ale jednocześnie prostym w implementacji sposobem oczekiwania na gotowość zasobów w systemach mikroprocesorowych. Aby poczekać na ustawienie TXE w trybie polling, możemy napisać taką oto pętlę `while` (tutaj dla odmiany używamy gotowego makra `__HAL_UART_GET_FLAG` do sprawdzania bitów statusowych):

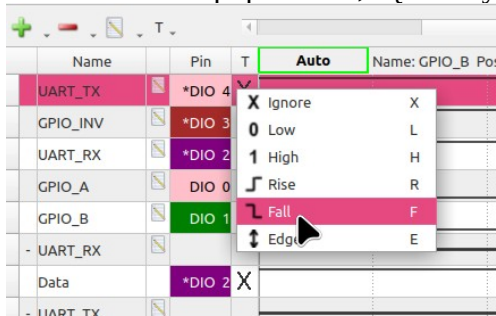
```
while(__HAL_UART_GET_FLAG(&huart6, UART_FLAG_TXE) == RESET){};
```

Oprócz sprawdzania flagi TXE, ta pętla nie robi zupełnie nic, marnując zasoby obliczeniowe procesora. I "kręci się" w jednym miejscu tak długo, jak flaga TXE jest skasowana, czyli tak długo, jak nie możemy wpisywać nowych danych do DR.

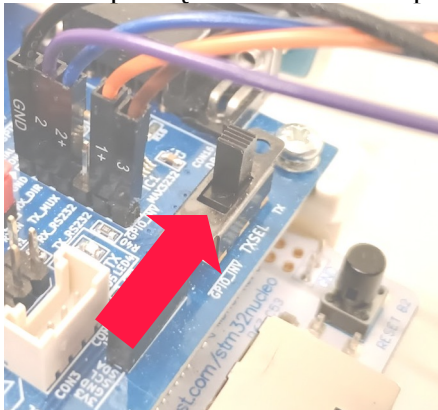
Sprawdźmy zatem, czy taka funkcja zadziała:

```
static void uart_tx_polling(char chr){  
    //czekamy na gotowość bitu TXE przed wpisaniem danych do DR  
    while(__HAL_UART_GET_FLAG(&huart6, UART_FLAG_TXE) == RESET){};  
  
    //wysyłamy znak (gdy już można)  
    huart6.Instance->DR = chr;  
}
```


Aby zaobserwować wysyłane znaki na terminalu, ustawmy tym razem wyzwalanie na kanale DIO4 analizatora. Jak poprzednio, będziemy wyzwalać przebieg zboczem opadającym:



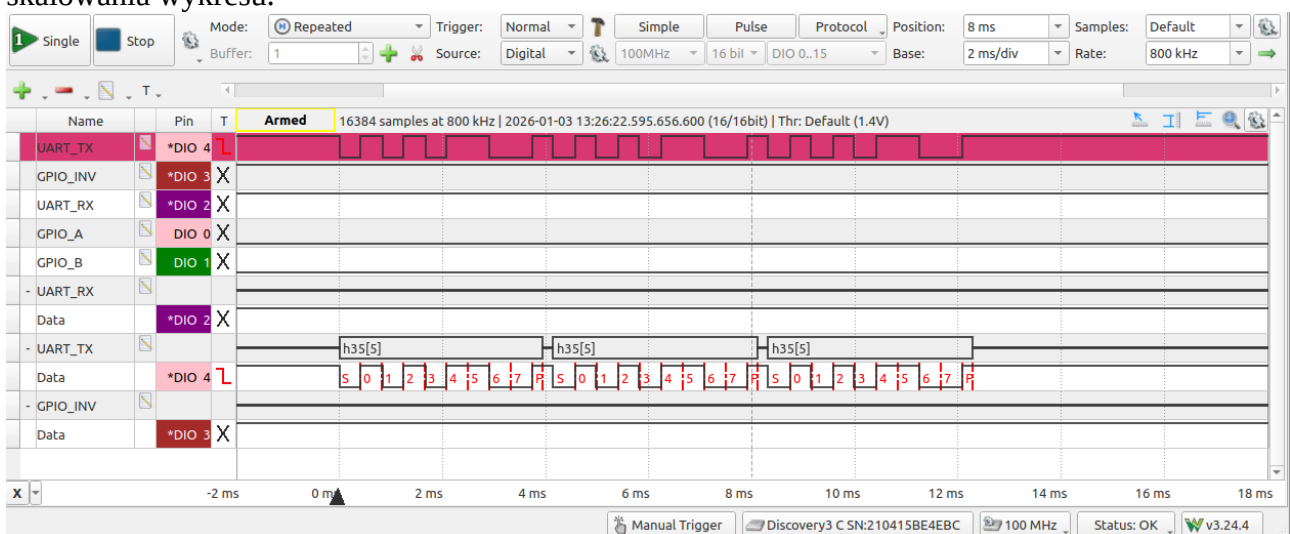
Uwaga: aby dało się zaobserwować wysyłane znaki na terminalu ttyS0, należy koniecznie ustawić przełącznik TXSEL do pozycji TX czyli "w prawo".



Cykl obserwacji może wyglądać np. tak:

- z terminala ttyACM0 wyślijmy np. znak '1', '2', lub '3', później robimy eksperymenty także z innymi znakami,
- obserwujemy, co się dzieje na przebiegach,
- sprawdzamy, czy na terminalu ttyS0 pojawia się po kilka kolejnych znaków '5',
- wysyłamy także inne znaki.

Przykładowy widok reakcji na wysłanie 3 znaków '5' (po wciśnięciu klawisza 3 w terminalu) przedstawiono poniżej. Uzyskanie takiego widoku może oczywiście wymagać przesuwania i skalowania wykresu.



3.2.5 Wysłanie danych przez UART6 z obserwacją zajętości bufora DR

Wiemy, że w trybie polling procesor w pustej pętli monitoruje flagę TXE przez co marnuje czas na oczekiwanie na gotowość rejestru DR. Ale sprawdźmy, ile tak naprawdę tego czasu marnuje i jak zachowuje się flaga TXE oraz rejestr DR. W tym celu zmodyfikujemy nieco funkcję `uart_tx_polling` w taki sposób, aby wyprowadzenie `GPIO_A` w możliwie dokładnie odzwierciedlało stan flagi TXE. Aby to uczynić, wystarczy zmienić pętlę oczekiwania w taki sposób, aby nie tylko sprawdzała stan TXE, ale także adekwatnie ustawiała `GPIO_A`. Dla ułatwienia, do ustawiania `GPIO_A` mamy przygotowaną w `main.h` makrodefinicję:

```
#define GPIO_A_H    {HAL_GPIO_WritePin(GPIO_A_GPIO_Port, GPIO_A_Pin, GPIO_PIN_SET);}
#define GPIO_A_L    {HAL_GPIO_WritePin(GPIO_A_GPIO_Port, GPIO_A_Pin, GPIO_PIN_RESET);}
```

Zmienimy pętlę oczekiwania w taki sposób, aby przy okazji ustawiała stan `GPIO_A`. Dla uzyskania adekwatnych obserwacji należy również "sklonować" kod oczekiwania do fragmentu po wpisaniu znaku do DR:

```
static void uart_tx_polling(char chr){
    //czekamy na gotowość bitu TXE przed wpisaniem danych do DR
    while(__HAL_UART_GET_FLAG(&huart6, UART_FLAG_TXE) == RESET){
        GPIO_A_L;
    }
    GPIO_A_H;

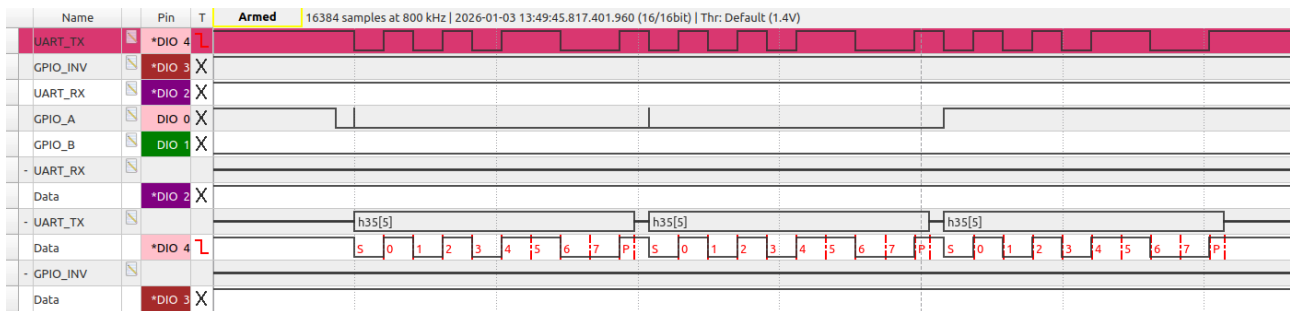
    //wysyłamy znak (gdy już można)
    huart6.Instance->DR = chr;

    //czekamy ponownie na gotowość bitu TXE
    //to działanie nadmiarowe, ale pozwoli na zaobserwowanie
    //co się dzieje z bitem TXE już po wpisaniu znaku do DR
    while(__HAL_UART_GET_FLAG(&huart6, UART_FLAG_TXE) == RESET){
        GPIO_A_L;
    }
    GPIO_A_H;
}
```

Wyślijmy teraz z terminala debuggera `ttyACM0` znaki '1', '2', '3' i '4'. Zaobserwujmy przebiegi. Udokumentujmy rezultaty na UPEL oraz zaprezentujmy działanie osobie prowadzącej - tylko uwaga: proszę najpierw zastanowić się i potrafić odpowiedzieć na pytanie, skąd bierze się takie zachowanie bitu TXE jak obserwowane na linii `GPIO_A`? W szczególności:

- Dlaczego linia TXE sygnalizuje gotowość do wpisania nowego znaku, gdy poprzednio wpisany znak dopiero zaczął być transmitowany? (widać to szczególnie dobrze przy pierwszym i ostatnim transmitowanym znaku)
- Czy w UART w STM32 mamy dostępny jakiś bit statusowy, który pozwalałby na sprawdzenie, kiedy rzeczywiście zakończyła się transmisja? (tj. kiedy UART skończył transmisję wszystkich bitów danych?) Który bit rejestru SR może służyć do tego celu? (hint: odpowiedź znajdziemy w treści wykładu dotyczącego IRQ i DMA)

Przykładowo, przy przesyłaniu trzech znaków '5' możemy otrzymać przebiegi przypominające poniższe:

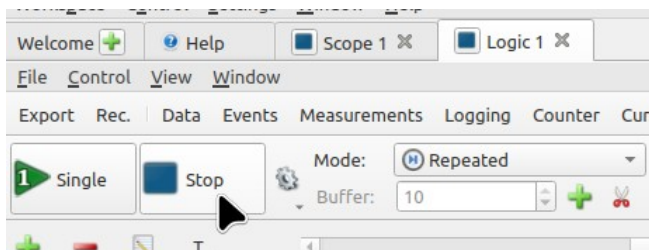


3.2.6 Obserwacja transmisji danych przez UART jako sygnału analogowego oraz dekodowanie przesyłanych danych

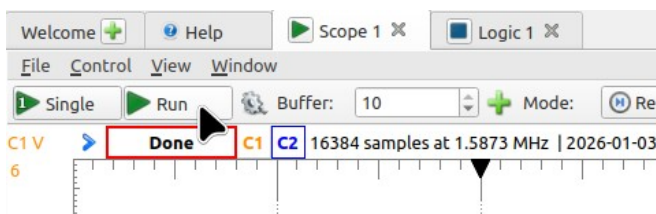
W tej części

- przyjrzymy się bliżej roli transceivera przy przesyłaniu sygnału,
- spróbujemy nauczyć się, w jaki sposób przesyłane są dane przez port szeregowy oraz
- spróbujemy zastanowić się i odpowiedzieć na pytanie, dlaczego osoby projektujące ten standard transmisji wybrały akurat taki sposób synchronizacji?

Zacznijmy od zatrzymania rejestracji sygnału cyfrowego klikając Stop w zakładce Logic.

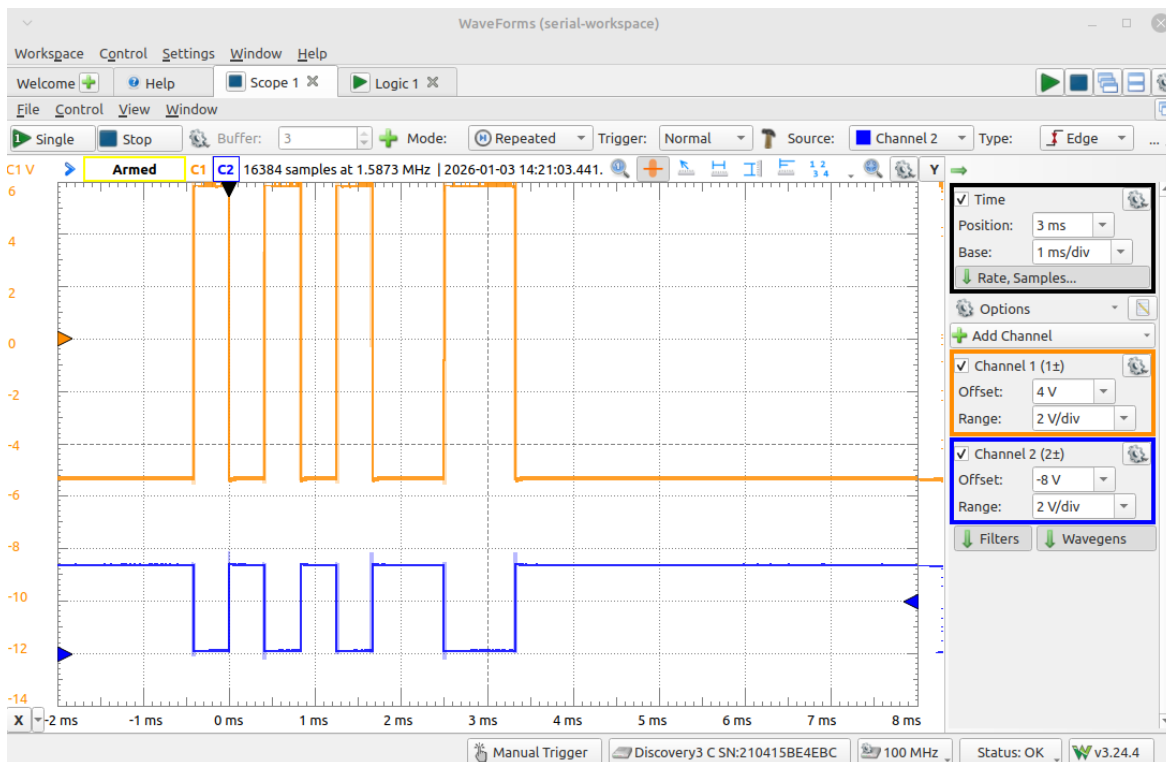


a następnie przejdźmy do zakładki Scope, gdzie na tym etapie powinien być gotowy do pracy oscyloskop (w razie problemów warto na chwilę wrócić do podpunktu 2.4.1 i sprawdzić ustawienia). W zakładce oscyloskopu wciskamy przycisk **Run**.



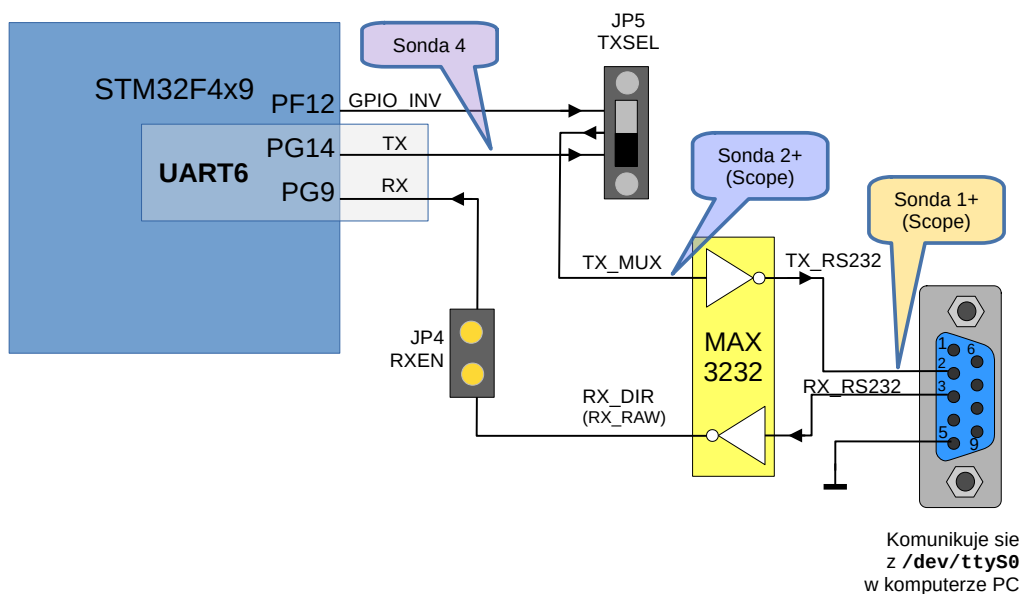
Spróbujmy wysłać pojedynczy znak na UART6 przez wysłanie '1' z terminala debuggera ttyACM0.

Po przesłaniu znaku, okienko oscyloskopu powinno wyglądać podobnie jak poniżej:



W kanale 1 na górze obszaru kreślenia mamy przebieg rejestrowany na linii TX interfejsu RS232-C (za transceiverem MAX3232). Natomiast na dole obszaru kreślenia widzimy przebieg rejestrowany na wyprowadzeniu TXD interfejsu UART6. Przydatny może być także do porównania przebieg cyfrowy rejestrowany przez sondę 4 - jednak, aby go zaobserwować, należy przełączyć się z powrotem do zakładki analizatora stanów logicznych Logic.

Fragment głównego schematu blokowego - dla przypomnienia:

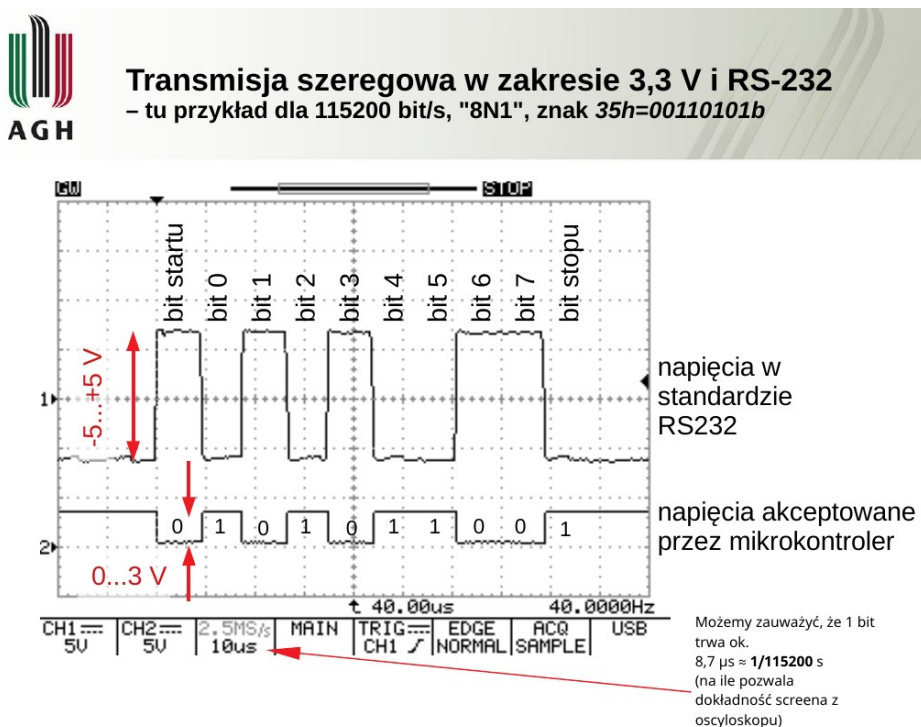


Aby uporządkować pracę, przed zgłoszeniem i wysłaniem rezultatów zadania, należy zmierzyć następujące parametry transmisji korzystając z modułu Scope i dostępnych w nim kursorów ekranowych (poniższy fragment dostępny jest na UPEL w wersji edytowalnej):

- Napięcia na TX_MUX w czasie:
 - przesyłania bitu "0": _____ V
 - przesyłania bitu "1": _____ V
- Napięcia na TX_RS232 w czasie:
 - przesyłania bitu "0": _____ V
 - przesyłania bitu "1": _____ V
- Czas trwania pojedynczego bitu: _____ μ s = _____ ms.
- Czas trwania całej ramki dla jednego znaku (od pierwszego zbocza bitu startu, do zakończenia bitu stopu): _____ ms.
- Za każdym razem
 - bit **startu** jest taki jak przesyłany stan logiczny: ____ (0 czy 1?)
 - bit **stopu** jest taki jak przesyłany stan logiczny: ____ (0 czy 1?)

Na tym etapie przydatny może okazać się jeden ze slajdów prezentacji z wykładu (temat "Interfejsy peryferyjne przewodowe w systemach IoT i wbudowanych", na którym zostały opisane przykładowe przebiegi czasowe przy transmisji UART).

O, ten:



Mając dostępne wszystkie dotychczas zestawione narzędzia, napisany kod i przeprowadzone pomiary, należy umieć odpowiedzieć na poniższe pytania oraz udokumentować na UPEL przeprowadzone eksperymenty.

- W jaki sposób kodowane są ramki transmisji interfejsu UART: który bit jest bitem startu, w jakiej kolejności przesyłane są dane, który bit jest bitem stopu?
- Jakie napięcia panują na liniach transmisji danych przed i za układem transceivera przy transmisji bitu "0" oraz bitu "1"?
- Ile czasu trwa przesyłanie jednego bitu w odniesieniu do ustawionej szybkości transmisji?
- Jakie są inne parametry transmisji przez UART i co zmieniają? Np.
 - Jak będzie wyglądał przebieg z więcej niż 1 bitem stopu, jak zachowuje się bit parzystości?
 - Czy można przysyłać mniejszą lub większą ilość danych w jednej ramce czy musi to być dokładnie 1 bajt czyli 8 bitów?
 - (można się tego nauczyć zarówno teoretycznie, na podstawie zewnętrznych źródeł, jak i praktycznie, wykonując eksperymenty - a najlepiej jedno i drugie ;))
- Jak będą wyglądały przebiegi czasowe, jeśli wyślemy dane bardziej nietypowe, np. pod rząd kilka bajtów 0x00, lub kilka bajtów 0xFF? Zastanówmy się czy i w jaki sposób typowa transmisja przez UART zabezpiecza nas przed utratą synchronizacji czasowej w takich przypadkach? Do przeprowadzenia eksperymentów może być przydatna edycja poniższego fragmentu kodu firmware (szczególnie zmiana znaku w `tx_chr`):

```
case '1':
case '2':
case '3':
case '4':
    int count = key - '0';
    const char tx_chr = 0x35;
    xprintf("sending %d chars of value %02X (%c)... ",count,(int)tx_chr,tx_chr);
    for(int i=0;i<count;i++){
        uart_tx_polling(tx_chr);
    }
    xprintf("done\n");
    break;
```

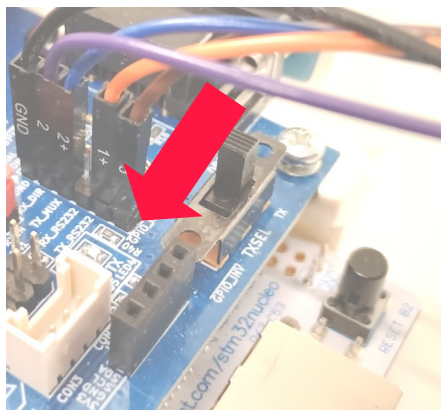
3.2.7 Implementacja programowa transmisji danych przez port szeregowy używając GPIO

Dotychczasowe doświadczenia z komunikacją przez UART, na tym etapie mogą pozwolić nam na podjęcie próby samodzielnej implementacji wysyłania ramki UART korzystając z wyprowadzenia GPIO oraz funkcji opóźniającej.

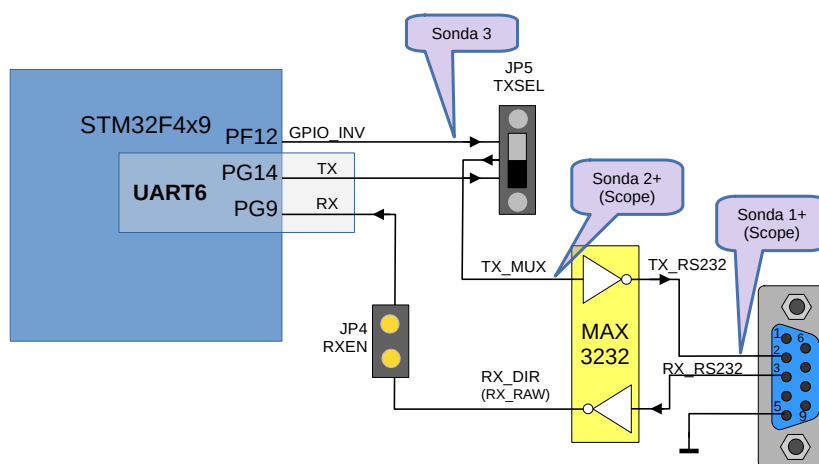
W ramach tego ćwiczenia proszę zatem spróbować zaimplementować funkcję wysyłającą 8-bitowy znak, jednak w sposób programowy, bez pomocy modułu UART, a jedynie z użyciem wyprowadzenia GPIO. Gotową implementację proszę udokumentować i przedstawić osobie prowadzącej.

Co mamy dostępne? Przede wszystkim możemy skorzystać z wyprowadzenia PF12, które w

aktualnej konfiguracji sprzętowej kontroluje sygnał **GPIO_INV**. Z kolei **GPIO_INV** jest skierowany do przełącznika TX_SEL. Do eksperymentów należy więc koniecznie ustawić przełącznik TX_SEL "w lewo", do pozycji **GPIO_INV**.



Możemy także swobodnie korzystać z modułu analizatora stanów logicznych oraz oscyloskopu w AD3 i np. porównywać przebiegi ze znanymi, wygenerowanymi uprzednio przy pomocy UART.



Od strony softwarowej, dokładne małe opóźnienia można realizować korzystając z funkcji **usdelay**. Funkcja ta pobiera na wejściu liczbę typu **uint16_t**:

```
void usdelay(uint16_t us)
```

Należy pamiętać, że realizacja krótkich opóźnień (rzędu pojedynczych mikrosekund) może nie być dokładna. Można przyjąć, że użyteczny zakres opóźnień zaczyna się od ok. 20 µs. Szczegóły implementacji tej funkcji znajdziemy w **us_delay.c / .h**.

Możemy również korzystać z gotowych makrodefinicji ustawiających wyprowadzenie **GPIO_INV** w stan wysoki (H) oraz niski (L): **GPIO_INV_H** oraz **GPIO_INV_L**, zdefiniowanych w **main.h**:

```
#define GPIO_INV_H {HAL_GPIO_WritePin(GPIO_INV_GPIO_Port, GPIO_INV_Pin, GPIO_PIN_SET);}
#define GPIO_INV_L {HAL_GPIO_WritePin(GPIO_INV_GPIO_Port, GPIO_INV_Pin, GPIO_PIN_RESET);}
```

Makrodefinicje te są widoczne globalnie w całym projekcie, w tym oczywiście w pliku **main.c**.

3.2.8 Klasa USB-CDC i minimalna implementacja adaptera USB-Serial (część nieobowiązkowa - dla osób zainteresowanych)

W oprogramowaniu do ćwiczenia mamy skonfigurowany kod realizujący funkcjonalność klasy CDD USB. Większość kodu stanowiąca sterownik CDC została wygenerowana przy pomocy aplikacji STM32CubeMX na podstawie wprowadzonej konfiguracji. Port USB, który został oprogramowany to port USB-OTG znajdujący się na dole płytki.

Dlatego na tym etapie można podłączyć przewód USB dla tego portu do komputera. Teraz szczególnie warto sprawdzać na którym numerze portu będzie zgłaszał się sterownik CDC z STM32 (oczywiście pamiętamy odpowiednie wywołanie komendy ls ;)).

Tutaj możemy w prosty sposób wypróbować interfejs programowy tego sterownika. Gdy wszystko jest poprawnie skonfigurowane i nie musimy wnikać w niskopoziomowe działanie sterownika USB, to obsługa odbierania i wysyłania danych nie jest skomplikowana. Do dalszych analiz warto otworzyć plik `./USB_DEVICE/App/usbd_cdc_if.c`.

Gdy sterownik odbierze jakieś dane od hosta USB (np. z dołączonego do komputera) to wywoła on funkcję `CDC_Receive_FS` z pliku `usbd_cdc_if.c`.

```
static int8_t CDC_Receive_FS(uint8_t* Buf, uint32_t *Len)
{
    /* USER CODE BEGIN 6 */

    //tutaj wpisujemy kod obsługi nadchodzących danych

    USBD_CDC_SetRxBuffer(&hUsbDeviceFS, &Buf[0]);
    USBD_CDC_ReceivePacket(&hUsbDeviceFS);
    return (USBD_OK);
    /* USER CODE END 6 */
}
```

Widzimy, że po zakończeniu obsługi nadchodzących wywoływane są funkcje przygotowujące bufor na kolejne dane oraz wznowiające odbiór danych. W ramach tworzenia własnego kodu można spróbować np. wyświetlić nadchodzące dane lub wywołać funkcję callback, np. `cdc_rxed_cb`:

```
extern void cdc_rxed_cb(uint8_t* buf, uint16_t len);
```

Funkcja ta jest od razu zadeklarowana jako extern w module `usbd_cdc_if.c` oraz jej definicja znajduje się w funkcji `main.c`. Alternatywnie, jeśli wystarczy nam przeprowadzenie prostego eksperymentu, można po prostu wyświetlić zawartość bufora, który został wskazany w wywołaniu `CDC_Receive_FS`.

Wysyłanie danych przez port szeregowy implementowany w klasie CDC jest nawet jeszcze prostsze. Wystarczy wywołać funkcję:

```
uint8_t CDC_Transmit_FS(uint8_t* Buf, uint16_t Len)
```

a po wysłaniu danych sterownik wywoła funkcję callback informującą o zakończeniu transmisji: `CDC_TransmitCplt_FS`.

4 Zagadnienia

Wymienione w niniejszej sekcji zagadnienia podsumowujące mają służyć do utrwalenia zdobytej wiedzy i umiejętności oraz do ułatwienia przygotowania się do sprawdzianu. Zagadnienia można opracować samodzielnie na podstawie dostępnych materiałów i doświadczeń zdobytych na laboratorium.

Hint: dla bardziej świadomego przygotowania się do sprawdzianu, warto również zajrzeć do materiałów wykładowych:

- Interfejsy embedded, IoT, część dotycząca UART
- Model programowy - układy peryferyjne, w szczególności slajdy
- DMA - część dotycząca działania interfejsów szeregowych

Zagadnienia warte powtórzenia:

- Zagadnienia zawarte w instrukcji do ćwiczenia i badane w czasie realizacji ćwiczenia,
- Podobieństwa i różnice pomiędzy modułami UART a USART w mikrokontrolerach ogólnego przeznaczenia:
 - Który jest synchroniczny, a który asynchroniczny?
 - Który był badany na laboratorium?
- Jaki jest minimalny zbiór sygnałów dla UART, a jaki dla USART zakładając możliwość nadawania i odbioru danych w trybie full-duplex?
- Znajomość mechanizmów działania modułu UART/USART:
 - sposobu wysyłania znaku
 - sposobu wysyłania wielu znaków jeden po drugim,
 - sposobu odbierania znaku,
- Format ramki UART/RS-232,
 - bity: startu, danych, stopu – kolejność, typowe opcje formatów ramek (zapis np. 8N1), m.in. w jakiej kolejności przesyłane są bity: startu, danych, parzystości, stopu.
 - zależności czasowe, głównie czasu trwania bitu danych od szybkości transmisji, np.
 - umiejętność obliczenia czasu trwania bitu na podstawie zadanej szybkości transmisji w bitach/s lub baudach lub ile czasu będzie trwać transmisja całej ramki jednego znaku przy zadanym formacie przesyłania znaku (np. 115200 baud, 8N1). Uwaga:
 - warto powtórzyć sobie np. ile sekund to milisekunda, a ile to mikrosekunda.
 - zakresy napięć portu szeregowego: po stronie UART mikrokontrolera "przed transceiverem" oraz po stronie interfejsu RS232 (tj. "po stronie kabla RS232" lub "za transceiverem")

- na podstawie obserwacji na laboratorium umiejętność wskazania po której stronie transceivera mamy typowy sygnał cyfrowy (np. 0-3,3V lub 0-5V), a po której sygnał symetryczny względem masy i o większej amplitudzie.
- Obsługa komunikacji przez port szeregowy:
 - czekanie na zwolnienie bufora nadawczego,
 - czekanie za zakończenie transmisji,
 - czekanie na nadejście znaku.